Claude Fable 5.1

Anthropic の**最も高性能な一般提供モデル**

`claude-fable-5-1`

一言でいうと

最高性能。ただし高価で、扱いに固有の作法がある。

- 最も要求の厳しい推論と、長時間のエージェント作業のためのモデル
- Claude Fable 5 の後継。同じティア・同じ価格
- 通常のデフォルトは Opus 5。**指名されたときだけ**使う
- 兄弟モデル Claude Mythos 5.1 は能力・価格・API が同一

基本スペック

項目	値
モデルID	`claude-fable-5-1`
コンテキスト	1M トークン（最大値がデフォルト）
最大出力	128K トークン
入力価格	$10.00 / 1M トークン
出力価格	$50.00 / 1M トークン
キャッシュ読込	$0.25 / 1M トークン
トークナイザ	Opus 4.8 / 4.7 と同一

価格：Opus の2倍

モデル	入力 $/1M	出力 $/1M
Claude Fable 5.1	**$10.00**	**$50.00**
Claude Opus 5	$5.00	$25.00
Claude Sonnet 5	$2.00	$10.00
Claude Haiku 4.5	$1.00	$5.00

入出力ともに Opus の2倍。**最高性能と引き換えのコスト**になる。

いつ使う / いつ使わない

使う

- 最高難度の推論
- 長ホライズンのエージェント作業
- 正しさがコストより重要

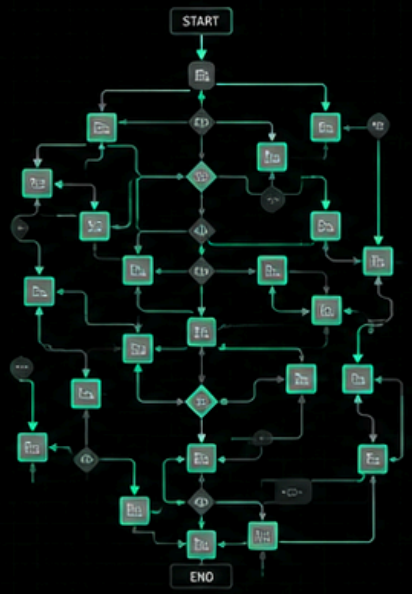
使わない

- 日常のコーディング・チャット
- コスト最適化が要件のとき

使う

最高性能が要るとき

Fable 5.1




```
[INFO] Fable 5.1 agent started
12:01:03  Planning complex task...
12:01:04  Analyzing codebase...
12:01:05  Building dependency graph...
12:01:06  [TOOL] read_file(path="/src/main.py") ✓
12:01:06  [TOOL] list_dir(path="/src") ✓
12:01:07  [TOOL] grep(pattern="class *A") ✓
12:01:08  [TOOL] read_file(path="/src/utils.py") ✓
12:01:09  [TOOL] read_file(path="/docs/arch.md") ✓
12:01:10  [TOOL] search_docs(query="auth flow") ✓
12:01:11  [TOOL] run_shell(cmd="pytest -q") ✓
12:01:15  [TOOL] run_shell(cmd="mypy .") ✓
12:01:17  [TOOL] run_shell(cmd="ruff check .") ✓
12:01:18  [TOOL] read_file(path="/tests/test_auth.py") ✓
    ⋮                ⋮
12:18:42  [TOOL] run_shell(cmd="alembic upgrade head") ✓
12:18:45  [TOOL] run_shell(cmd="pytest -q") ✓
12:18:52  [TOOL] write_file(path="/src/auth/service.py") ✓
12:18:53  [TOOL] write_file(path="/src/auth/repo.py") ✓
12:18:55  [TOOL] update_docs(file="/docs/arch.md") ✓
12:19:01  [TOOL] run_shell(cmd="pytest -q") ✓
12:19:05  [TOOL] run_shell(cmd="ruff check .") ✓
12:19:07  [INFO] All checks passed
12:19:07  [INFO] Task completed successfully ✓
```

Solving complex workflow... 87%

- 最高難度の推論
- 長時間のエージェント作業
- 正しさがコストより重要



価格は Opus の2倍

使わない

Opus 5 や Sonnet 5 で足りる

チャット画面 ● オンライン

あなた 10:30
この関数のバグを直して、テストも追加して。

AI (Opus 5 / Sonnet 5) 10:31
修正とテストを追加しました。以下が変更点です。
- 境界条件の不具合を修正
- ユニットテストを追加（3件）
- すべてのテストが通過しました

分類処理バッチ画面 ● 完了

ID	テキスト	カテゴリ	信頼度	ステータス
1001	この製品は使いやすいです	ポジティブ	0.92	完了
1002	配送が遅れて残念でした	ネガティブ	0.89	完了
1003	価格に見合った品質です	ニュートラル	0.85	完了
…	…	…	…	…

処理件数： 10,000 / 10,000件 100%

- 日常のコーディング
- チャット・分類
- コスト最適化が要件

おさえるべき4つの特徴

1. 思考が**常時オン**
2. 生の思考過程は**返らない**
3. **拒否**への対応が要る
4. Fable 5 からの**破壊的変更**

どれも旧モデル向けのコードを壊す。

Claude Fable 5.1 の4つの特徴

1



思考が常時オン

オフにできない。深さは effort で調整

2



生の思考は返らない

既定は omitted。要約は summarized で

3



拒否への対応が要る

HTTP 200 で返る。stop_reason を確認

4



破壊的変更がある

Fable 5 のコードはそのままでは壊れる

特徴1：思考が常時オン

`thinking` は**省略する**か `{type: "adaptive"}` を送る。

指定	結果
省略 / `adaptive`	✅ 常時オンで動く
`{type: "disabled"}`	❌ 400 エラー
`{type: "enabled", budget_tokens: N}`	❌ 400 エラー

「思考をオフ」も「トークン予算で制御」も**もう使えない**。

深さは effort で決める

`output_config.effort` の5段階。デフォルトは `high` 。

レベル	使いどころ
`low` / `medium`	日常的な作業。多くはこれで足りる
`high`	既定値。品質とコストのバランス
`xhigh`	コーディング・エージェント作業
`max`	正しさがコストより重要なとき

新しいモデルは**低い effort でも旧モデルの高い effort を上回る**ことがある。

特徴2：生の思考過程は返らない

どの設定でも生のテキストは外に出ない。表示は `thinking.display` で選ぶ。

設定	挙動
`omitted`（既定）	中身は空文字
`summarized`	読める要約が返る
`updates`（ベータ）	ツール呼び出しの合間の進捗メモ

既定が `omitted` のため、指定しないと**長い沈黙の後に答えが出る UI** になる。

特徴3：拒否（refusal）への対応

安全性の分類器がリクエストを拒否することがある。

- HTTP は **200 で返る**（エラーにならない）
- `stop_reason` が `refusal` になる
- `content` を読む**前に** `stop_reason` を必ず確認する

例外が飛ばないため、素直に書くと**黙って壊れる**。

フォールバックを既定で入れる

```
betas: ["server-side-fallback-2026-07-01"]
fallbacks: "default"
```

- 拒否のカテゴリに応じて**自動で別モデルへ回る**
- モデルのリストを自分で管理しなくてよい
- 出力前の拒否は課金されない

特徴4：Fable 5 からの破壊的変更

Fable 5 のコードは**そのままでは壊れる**。

1. 強制ツール利用が廃止
2. thinking がモデルに紐づく
3. 履歴編集で thinking が無効に

Fable 5 → 5.1 の破壊的変更

1 強制ツール利用が廃止

tool_choice: any は非推奨に。
強制利用できなくなりました。

Before（5.0）

```
{
  "tool_choice": "any"
}
```

400 Bad Request

After（5.1）

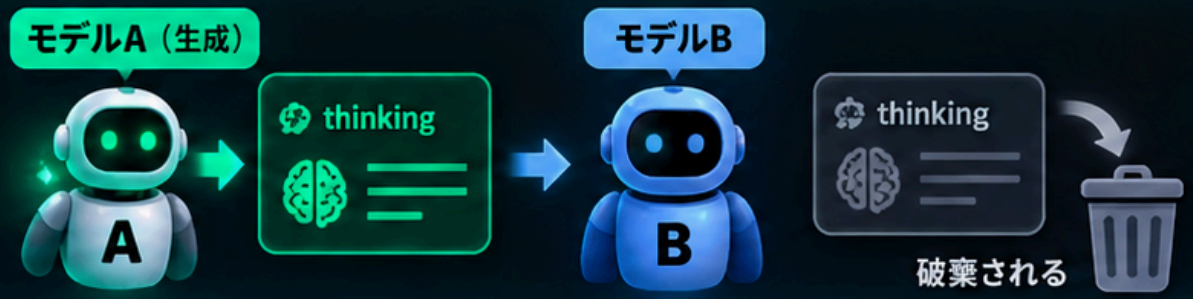
```
{
  "tool_choice": "auto"
}
```

対処法

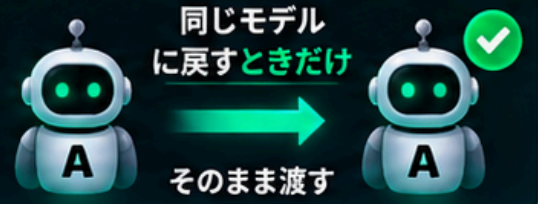
- プロンプトで ツール名を指示
- または
- { } 構造化出力

2 thinking がモデルに紐づく

thinking ブロックは生成した
モデル専用になりました。



対処法

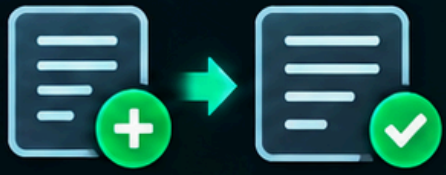


3 履歴編集で thinking が無効に

過去のメッセージを書き換えると、
後続の thinking ブロックが
すべて無効になります。



対処法



ハーネスは **追記のみ** にする

破壊的変更 ①②

① 強制ツール利用が廃止

`tool_choice` の `any` / `tool` は 400。`auto` + プロンプト指示、`strict: true`、または構造化出力に置き換える。

② thinking がモデルに紐づく

生成したモデル以外に渡すと**黙って破棄**される（課金もされない）。

③ 履歴は追記のみにする

preserved thinking — 過去のターンを編集すると thinking ブロックが無効になる。

- ハーネスは**追記のみ**で設計する
- 2026-08-31 以降に作られたアカウントは編集済み履歴で **400**
- 後発モデルでは全アカウントに適用される

圧縮や履歴の書き換えを行う実装は**要見直し**。

その他の制約

項目	内容
プレフィル	アシスタントのプレフィルは 400
データ保持	30日保持が必須。ZDR では原則使えない
Priority Tier	非対応。指定すると失敗する
1ターンの長さ	数分かかることがある
プロンプト	細かすぎる指示は品質を下げる

Fable 5.1 で増えたもの

- **メッセージ単位の effort 指定**（ベータ）— 途中で変えてもキャッシュが飛ばない
- **ターン限定のシステムメッセージ**（ベータ）— 1ターンだけ有効な指示
- **進捗メモ** `thinking.display: "updates"`（ベータ）
- **キャッシュ読込 $0.25/1M**
- **コンテンツの来歴（provenance）**

まとめ

1. **常時思考オン**。深さは `effort` で調整する
2. **拒否をハンドリング**し、フォールバックを入れる
3. **履歴は追記のみ**にする
4. 通常は Opus 5。**指名されたときだけ** Fable 5.1

> 最高性能だが高価。作法を知らないと黙って壊れる。